ASSIGNMENT-4

Team Members:

Tamanna Sharma – 20251502034

Siddhi Vyas – 20251501163

Pranjali Nama – 20251502022

Krish Kumar- 20251502015

Shobhit Rai- 20251502032

Object-Oriented Programming Lab Assignment

Inheritance and Friend Class-

Problem 1 — Inheritance: Employee Salary Calculation

A company maintains basic information about its employees and their monthly salary components. The basic employee

information is common to all employees, while the final salary calculation depends on additional information

maintained for a full-time employee.

Write a C++ program using inheritance to model this situation. The program should accept an employee's ID, name,

basic salary, and allowance percentage. Calculate the allowance and gross salary, and display the employee details

along with the calculated gross salary.

The common employee information and functionality should be placed in an appropriate base class, while the

additional information and salary-calculation functionality should be implemented in a derived class.

Requirements

- **Create a suitable base class to store the employee ID, name, and basic salary.**
- **Create a derived class that stores the allowance percentage and calculates the allowance and gross salary.**
- **Reuse the base-class members/functionality in the derived class.**
- **Display the employee ID, name, basic salary, allowance, and gross salary.**
- **Use a suitable access specifier so that the derived class can use the required base-class data without making the**

data unnecessarily public.

Input

- **Employee ID (integer)**
- **Employee name (single word)**
- **Basic salary (floating-point number)**
- **Allowance percentage (floating-point number)**

Output

- **Employee ID and name**
- **Basic salary**
- **Calculated allowance**
- **Gross salary = basic salary + allowance**

Constraints

- **Basic salary > 0**
- **0 ≤ allowance percentage ≤ 100**

Code:

```cpp
# include<bits/stdc++.h>
using namespace std;

class Employee{
    protected:
    int employeeID;
    string name;
    float basicSalary;

        public:
        void getEmployeeDetails(){
            cout<<"Enter Employee ID: ";
            cin>>employeeID;
            cout<<"Enter Employee Name:";
            cin.ignore();
            getline(cin, name);
            cout<<"Enter basic salary:";
            cin>>basicSalary;
        }
     };

     class FullTimeEmployee: public Employee{
        private:
        float allowancePercentage;
        float allowance;
        float grossSalary;

        public:
        void calculateSalary(){
            cout<<"Enter allowance percentage:";
            cin>>allowancePercentage;

            allowance = basicSalary*allowancePercentage/100;
            grossSalary = basicSalary+ allowance;
        }

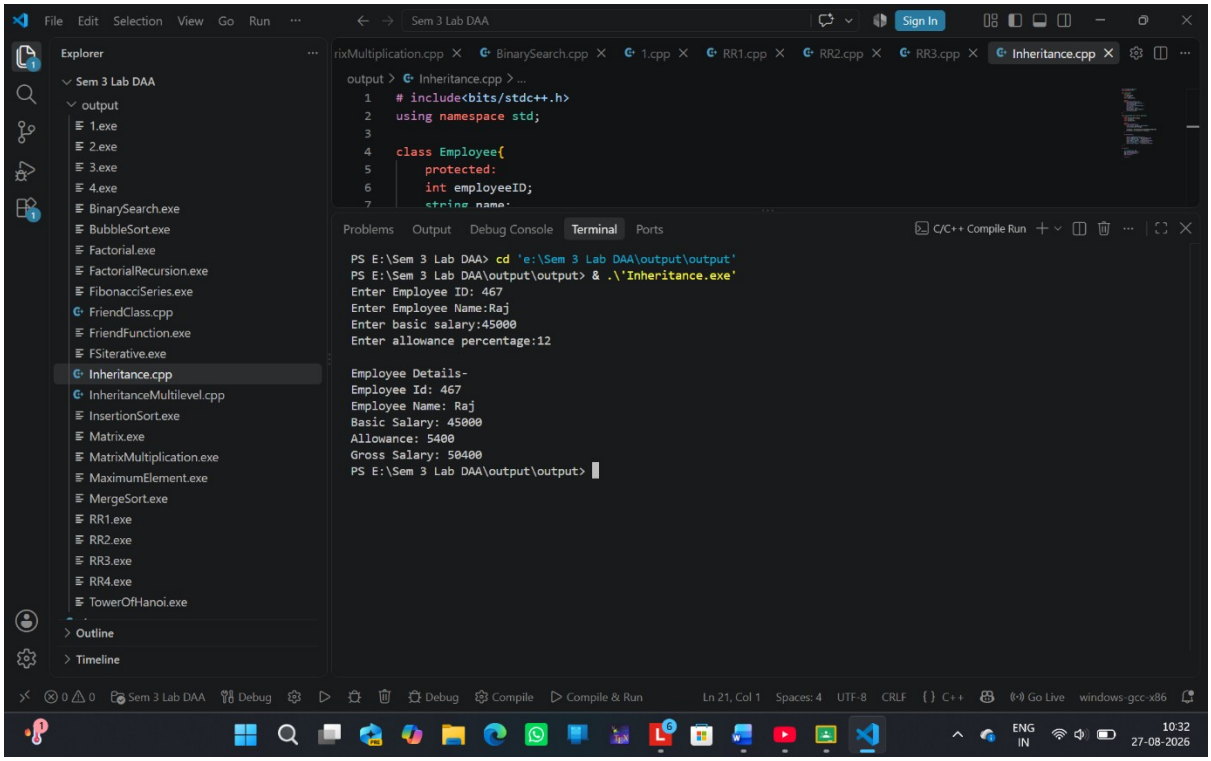
        void display(){

            cout<<"\nEmployee Details-\n";
            cout<<"Employee Id: "<<employeeID<<endl;
            cout<<"Employee Name: "<<name<<endl;
            cout<<"Basic Salary: "<<basicSalary<<endl;
            cout<<"Allowance: "<<allowance<<endl;
            cout<<"Gross Salary: "<<grossSalary<<endl;
        }
     };

     int main(){
```

```
49.
50.         FullTimeEmployee emp;
51.         emp.getEmployeeDetails();
52.         emp.calculateSalary();
53.         emp.display();
54.
55.         return 0;
56.
57.     }
```

Output:



Problem 2 — Inheritance: Academic Performance through Multilevel Inheritance

A college wants to generate a simple academic performance report for a student. The student's personal information is

maintained at one level, marks in theory subjects are added at the next level, and the final percentage and performance

category are determined at a further level.

Write a C++ program using multilevel inheritance to represent this process. Accept the student's roll number, name,

marks in three subjects, and calculate the total marks, percentage, and performance category.

Use the inheritance hierarchy meaningfully so that information introduced at an earlier level is reused by the

subsequent derived class.

Requirements

¬ Create a base class for student identity information such as roll number and name.

¬ Derive a second class that stores marks in three subjects and calculates the total marks.

¬ Derive a third class that calculates the percentage and determines the performance category.

¬ Display the student's details, subject marks, total, percentage, and category.

¬ Use the inherited members/functions from the earlier levels instead of duplicating the same data.

Input

¬ Roll number (integer)

¬ Student name (single word)

¬ Marks in three subjects (each an integer)

Output

¬ Roll number and name

¬ Three subject marks

¬ Total marks out of 300

¬ Percentage

¬ Performance category using these rules:

¬ 90% and above: Excellent

- 75% to below 90%: Very Good
- 60% to below 75%: Good
- 40% to below 60%: Average
- Below 40%: Needs Improvement

Constraints

- Each subject mark is between 0 and 100.
- The student name is a single word.

Code:

```
# include <bits/stdc++.h>
using namespace std;

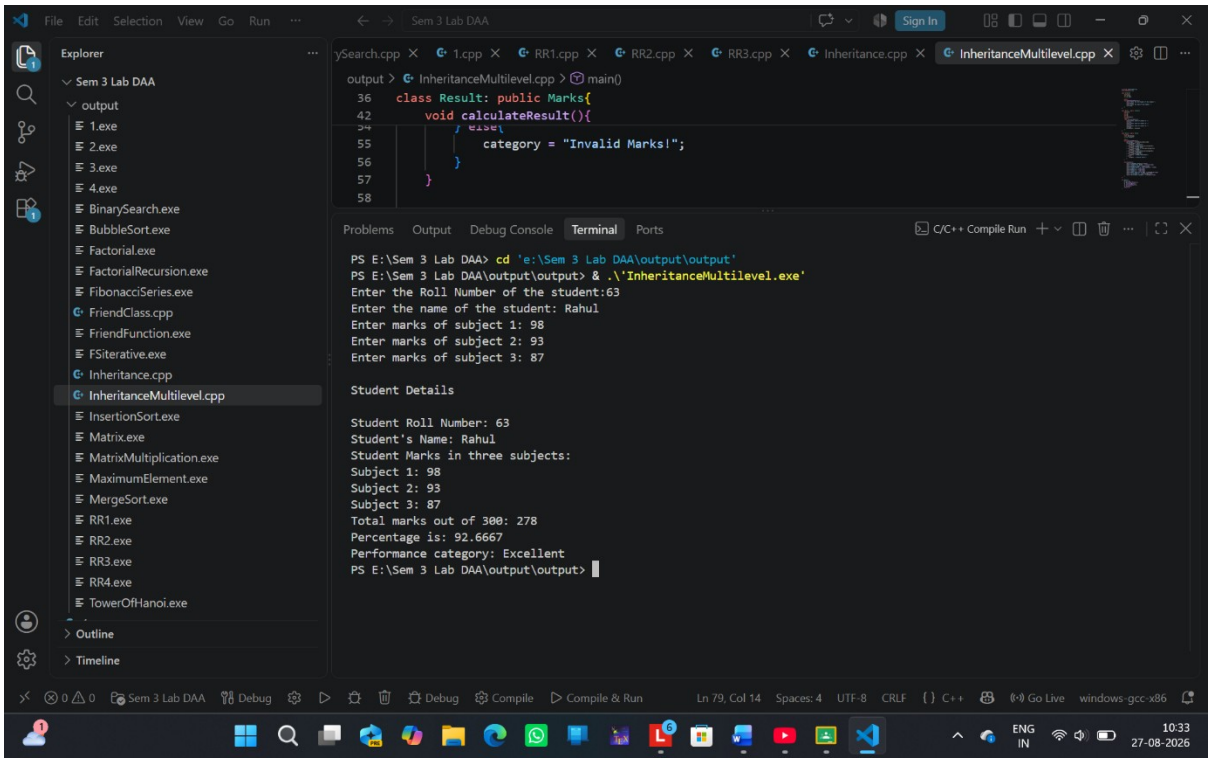
class Student{
    protected:
    int rollno;
    string name;

    public:
         void getStudentDetails(){
             cout<<"Enter the Roll Number of the student:";
             cin>>rollno;
             cout<<"Enter the name of the student: ";
             cin>>name;
         }
     };

     class Marks : public Student{
         protected:
         int m1;
         int m2;
         int m3;
         int totalMarks;
          public:
          void calculateMarks(){
             cout<<"Enter marks of subject 1: ";
             cin>>m1;
             cout<<"Enter marks of subject 2: ";
             cin>>m2;
             cout<<"Enter marks of subject 3: ";
             cin>>m3;
             totalMarks = m1+m2+m3;
```

```
33.          }
34.      };
35.
36.      class Result: public Marks{
37.          private:
38.          float percentage;
39.          string category;
40.
41.          public:
42.          void calculateResult(){
43.              percentage = (totalMarks/300.0)*100;
44.              if(percentage >= 90){
45.                  category = "Excellent";
46.              } else if(percentage>=75 && percentage<90){
47.                  category = "Very Good";
48.              } else if(percentage >= 60 && percentage<75){
49.                  category = "Good";
50.              } else if(percentage>=40 && percentage<60){
51.                  category = "Average";
52.              } else if(percentage<40){
53.                  category = "Needs Improvement";
54.              } else{
55.                  category = "Invalid Marks!";
56.              }
57.          }
58.
59.          void display(){
60.              cout<<"\nStudent Details\n"<<endl;
61.              cout<<"Student Roll Number: "<<rollno<<endl;
62.              cout<<"Student's Name: "<<name<<endl;
63.              cout<<"Student Marks in three subjects: "<<endl;
64.              cout<<"Subject 1: "<<m1<<endl;
65.              cout<<"Subject 2: "<<m2<<endl;
66.              cout<<"Subject 3: "<<m3<<endl;
67.              cout<<"Total marks out of 300: "<<totalMarks<<endl;
68.              cout<<"Percentage is: "<<percentage<<endl;
69.              cout<<"Performance category: "<<category<<endl;
70.          }
71.      };
72.
73.      int main(){
74.          Result s1;
75.          s1.getStudentDetails();
76.          s1.calculateMarks();
77.          s1.calculateResult();
78.          s1.display();
79.          return 0;
80.      }
```

Output:



Problem 3 — Friend Class: Bank Account Verification

A bank stores sensitive account information such as the account number, account holder's name, and current balance

inside an account object. A separate verification department must inspect this information to prepare a verification

report. For security reasons, the account data should remain private and should not be accessed directly from outside

the account class.

Write a C++ program in which a separate verification class is given friend-class access to the account class. The

verification class should use the account's private data to determine whether the account satisfies a minimum-balance

requirement and display an appropriate verification report.

Requirements

¬ʌ Create an Account class containing account number, account holder name, and balance as private data members.

¬ʌ Create a separate Verification class that is declared as a friend of Account.

¬ʌ The Verification class must directly use the private account data to perform the verification.

¬ʌ Accept the account details and a minimum balance required for verification.

¬ʌ Display the account details, minimum required balance, and whether the account is Verified or Not Verified.

¬ʌ Do not make the account's private data public merely to allow the verification class to access it.

Input

¬ʌ Account number (integer)

¬ʌ Account holder name (single word)

¬ʌ Current balance (floating-point number)

¬ʌ Minimum required balance (floating-point number)

Output

¬ʌ Account number and holder name

¬ʌ Current balance

¬ʌ Minimum required balance

¬ʌ Verification status: Verified if current balance is greater than or equal to the minimum required balance; otherwise

Not Verified

Constraints

¬ʌ Current balance ≥ 0

¬ؕ Minimum required balance ≥ 0

Code:

```
#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

class Verification;

class Account {
    private:
        int accountNumber;
        string accountHolderName;
        double currentBalance;

    public:
        Account(int accNum = 0, string name = "", double balance = 0.0)
            : accountNumber(accNum), accountHolderName(name), currentBalance(balance) {}

        // Member function to accept account details from user
        void acceptDetails() {
            cin >> accountNumber >> accountHolderName >> currentBalance;
        }

        friend class Verification;
    };

    class Verification {
    public:
        void verifyAccount(const Account& acc, double minRequiredBalance) {
            cout << fixed << setprecision(2);
            cout << "\n--- Account Verification Report ---\n";

            cout << "Account Number          : " << acc.accountNumber << "\n";
            cout << "Account Holder Name     : " << acc.accountHolderName << "\n";
            cout << "Current Balance         : $" << acc.currentBalance << "\n";
```

```
            cout << "Minimum Required Balance: $" << minRequiredBalance << "\n";

            if (acc.currentBalance >= minRequiredBalance) {
                cout << "Verification Status     : Verified\n";
            } else {
                cout << "Verification Status     : Not Verified\n";
            }
        }
    };

    int main() {
        Account myAccount;
        double minRequiredBalance;

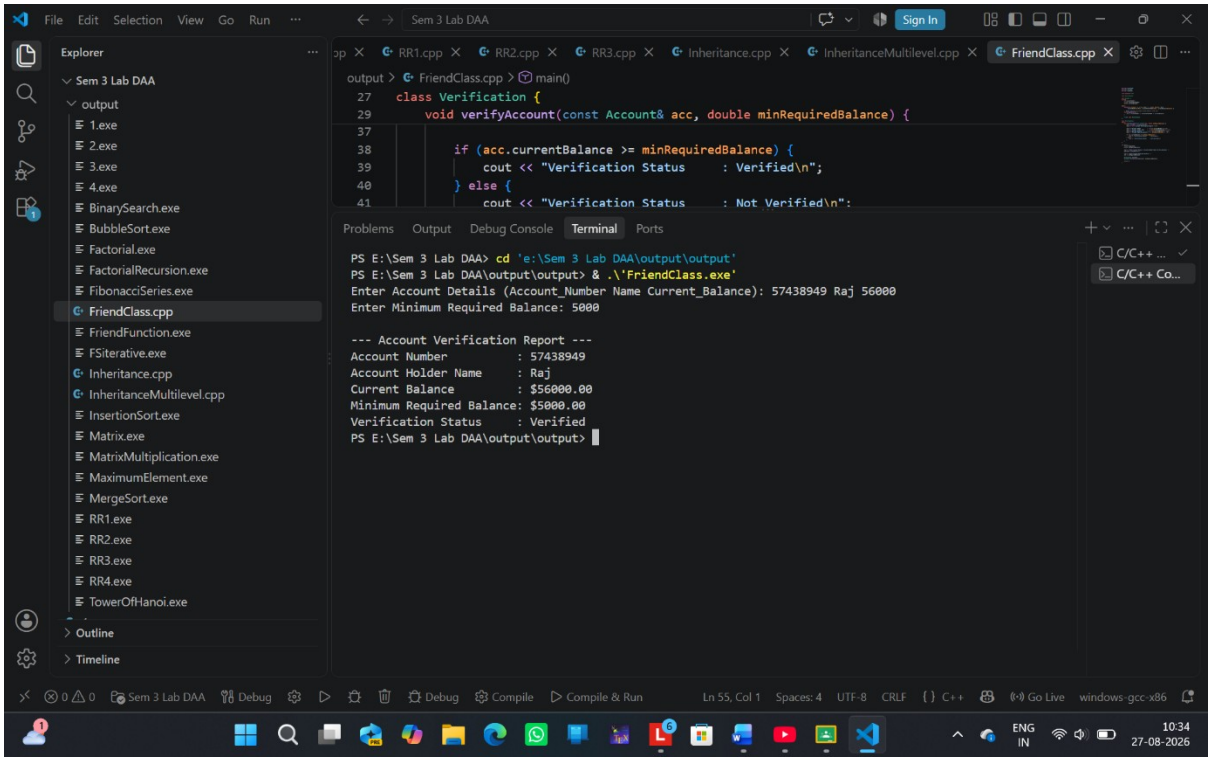
        cout << "Enter Account Details (Account_Number Name Current_Balance): ";
        myAccount.acceptDetails();

        cout << "Enter Minimum Required Balance: ";
        cin >> minRequiredBalance;

        Verification verifier;
        verifier.verifyAccount(myAccount, minRequiredBalance);

        return 0;
    }
```

Output:


File Edit Selection View Go Run
Sem 3 Lab DAA
Sign In
Explorer
Sem 3 Lab DAA
output
1.exe
2.exe
3.exe
4.exe
BinarySearch.exe
BubbleSort.exe
Factorial.exe
FactorialRecursion.exe
FibonacciSeries.exe
FriendClass.cpp
FriendFunction.exe
FSiterative.exe
Inheritance.cpp
InheritanceMultilevel.cpp
InsertionSort.exe
Matrix.exe
MatrixMultiplication.exe
MaximumElement.exe
MergeSort.exe
RR1.exe
RR2.exe
RR3.exe
RR4.exe
TowerOfHanoi.exe
Outline
Timeline
RR1.cpp
RR2.cpp
RR3.cpp
Inheritance.cpp
InheritanceMultilevel.cpp
FriendClass.cpp
output > FriendClass.cpp > main()
27 class Verification {
29 void verifyAccount(const Account& acc, double minRequiredBalance) {
37
38 if (acc.currentBalance >= minRequiredBalance) {
39 cout << "Verification Status : Verified\n";
40 } else {
41 cout << "Verification Status : Not Verified\n";
Problems Output Debug Console Terminal Ports
PS E:\Sem 3 Lab DAA> cd 'e:\Sem 3 Lab DAA\output\output'
PS E:\Sem 3 Lab DAA\output\output> & .\'FriendClass.exe'
Enter Account Details (Account_Number Name Current_Balance): 57438949 Raj 56000
Enter Minimum Required Balance: 5000
--- Account Verification Report ---
Account Number : 57438949
Account Holder Name : Raj
Current Balance : $56000.00
Minimum Required Balance: $5000.00
Verification Status : Verified
PS E:\Sem 3 Lab DAA\output\output>
C/C++ ...
C/C++ Co...
Sem 3 Lab DAA
Debug
Compile
Compile & Run
Ln 55, Col 1
Spaces: 4
UTF-8
CRLF
C++
Go Live
windows-gcc-x86
ENG
IN
10:34
27-08-2026